Temperature-Based Automatic Fan Control System Using Arduino

Submitted by: Narendhira Prasath
Submitted by: Narendhira Prasath
Date: June 2026

Introduction

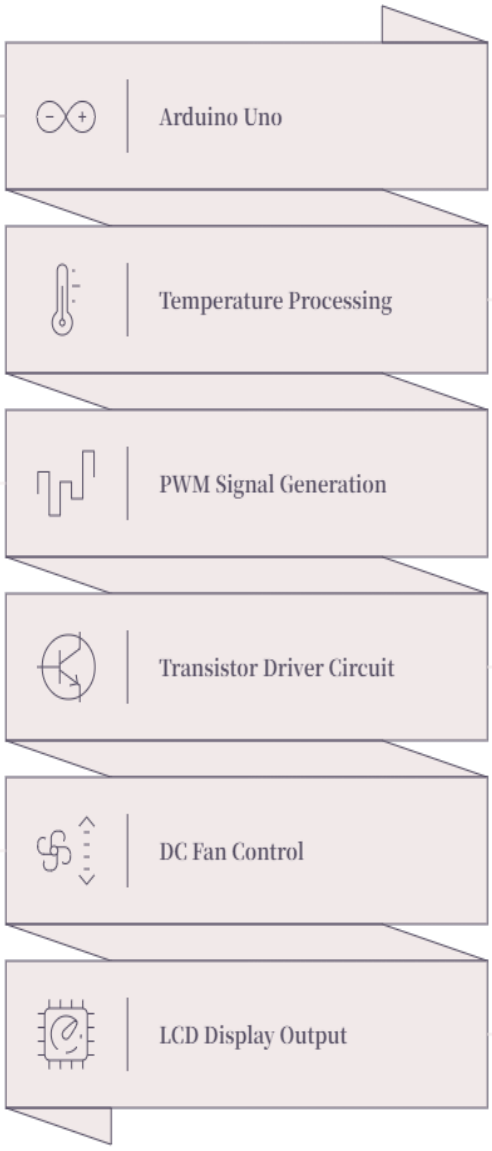
For this task I built a temperature-based automatic fan control system in Tinkercad — a small closed-loop system where a TMP36 sensor reads the surrounding temperature, an Arduino Uno decides how fast the fan should spin based on that reading, and a 16×2 LCD shows what's happening in real time. It's the same sense-process-actuate loop that shows up in real IoT systems, just running locally instead of talking to a cloud server, which made it a reasonable starting point for understanding the sensing layer before adding any wireless piece on top.

The simulation runs entirely in Tinkercad, no physical hardware involved. The goal was simple: read the temperature, switch fan speed automatically across three bands, and show both the temperature and fan status on the LCD without needing to monitor it manually.

Components Used

- Arduino Uno R3
- TMP36 analog temperature sensor
- 16×2 LCD display (LiquidCrystal library, parallel mode)
- DC motor (standing in for the fan)
- NPN transistor (motor switch/driver)
- 1N4007 flyback diode
- 1kΩ transistor base resistor, 220Ω LCD resistor
- Breadboard and jumper wires

ARDUINO FAN CONTROL SYSTEM



The central control unit for the system.

The Arduino reads temperature data.

The Arduino generates a Pulse Width Modulation signal.

The transistor amplifies the PWM signal.

The transistor controls the speed of the DC fan.

The Arduino displays the temperature and fan speed.

Circuit Design and Working Principle

The TMP36's output pin goes straight to A0 on the Arduino. It puts out a voltage that scales linearly with temperature (10mV per °C, with a 500mV offset at 0°C), so the Arduino just needs to read the analog value and run it through that conversion.

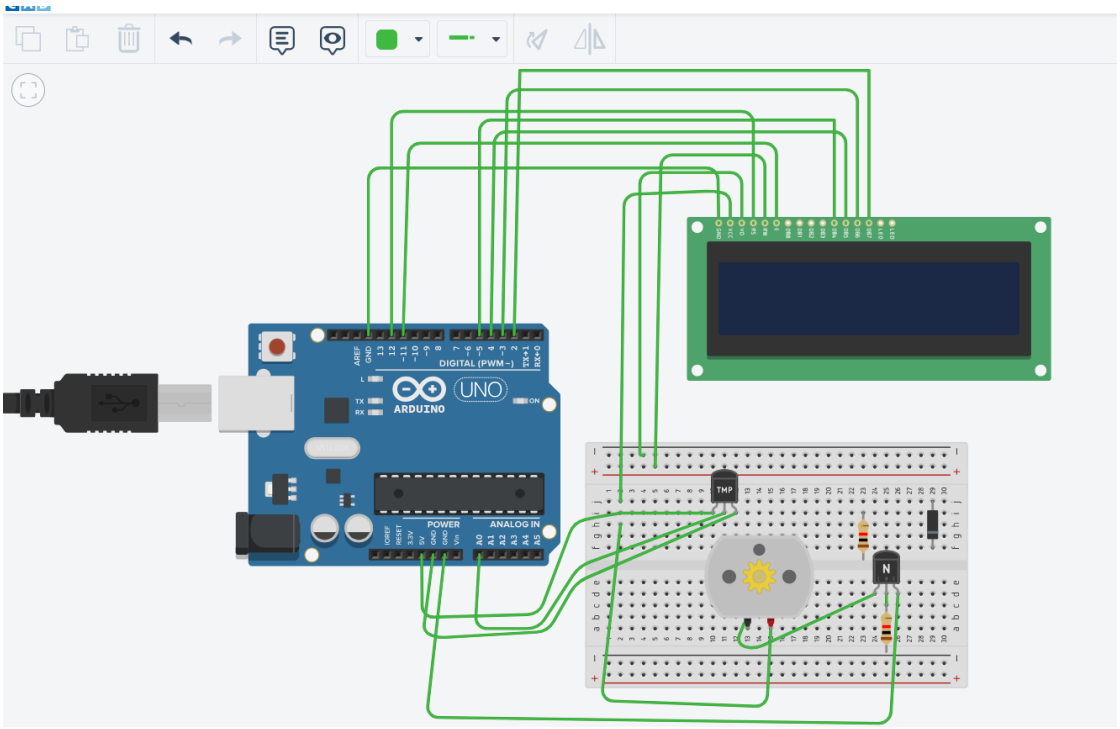


Figure 1. Tinkercad circuit layout — Arduino Uno, TMP36 sensor, NPN transistor driver, DC motor, and 16×2 LCD on a breadboard.

The fan itself isn't driven directly off an Arduino pin — a DC motor pulls more current than a digital output can safely supply. Pin 9 (PWM-capable) feeds the base of an NPN transistor through a 1k resistor, and the transistor does the actual switching for the motor circuit. A 1N4007 diode sits across the motor terminals to clamp the back-EMF spike that happens every time the motor switches off; without it, that spike can degrade the transistor over repeated cycles. The LCD runs in 6-pin mode (RS, EN, and four data lines), wired to D2–D5, D11, and D12, and just prints the current temperature and fan status every loop cycle.

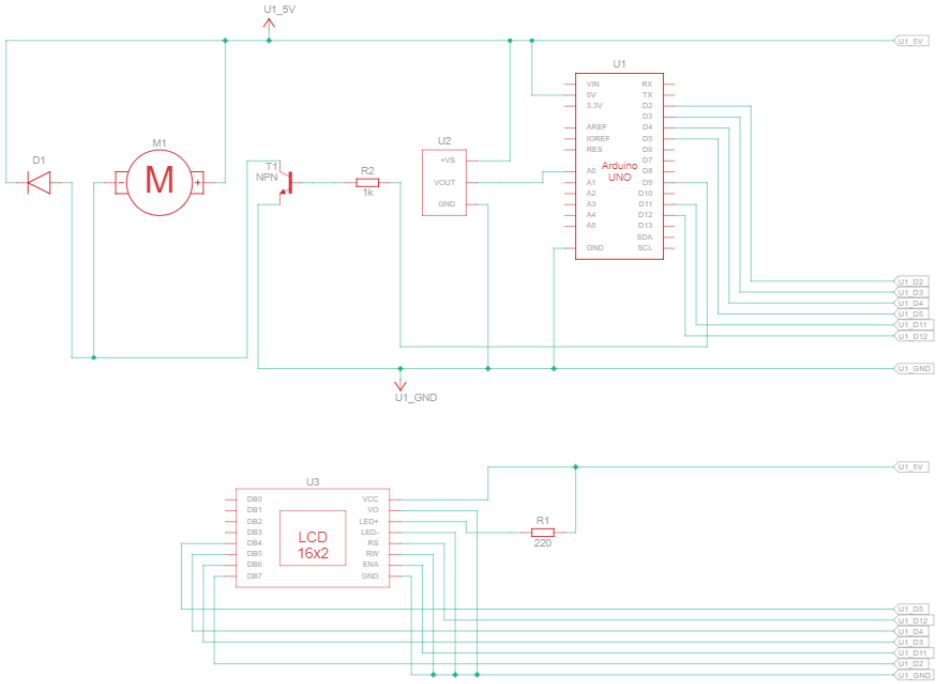


Figure 2. Schematic view exported from Tinkercad — U1 Arduino Uno, U2 TMP36, U3 16×2 LCD, M1 DC motor, D1 flyback diode, T1 NPN transistor.

Control Logic

The fan speed is decided by three fixed temperature bands:

Temperature Range	Fan Status	PWM Value
Below 20°C	OFF	0
20°C – 30°C	MEDIUM	150
Above 30°C	HIGH	255

Code Explanation

The core logic is small. After lcd.begin(16, 2) and a short “AUTO FAN CTRL / Initializing” splash screen, the main loop does four things every cycle:

- Reads the raw analog value from A0, converts it to voltage, then to °C using the standard TMP36 formula: temperatureC = (analogRead(tempPin) * 5.0 / 1024.0 − 0.5) * 100.0
- Compares that value against the three thresholds (below 20°C, 20–30°C, above 30°C) and sets fanSpeed and fanStatus accordingly.
- Calls analogWrite(motorPin, fanSpeed) — the actual PWM output the transistor circuit converts into motor speed.
- Updates the LCD and prints the same reading to the Serial Monitor for debugging.

It's a deliberately simple state machine — no hysteresis, no debouncing — which is fine for a simulation but is one of the first things worth adding before running this on real hardware, since a sensor hovering right at a threshold (say 29.9–30.1°C) would otherwise make the fan flicker between MEDIUM and HIGH.

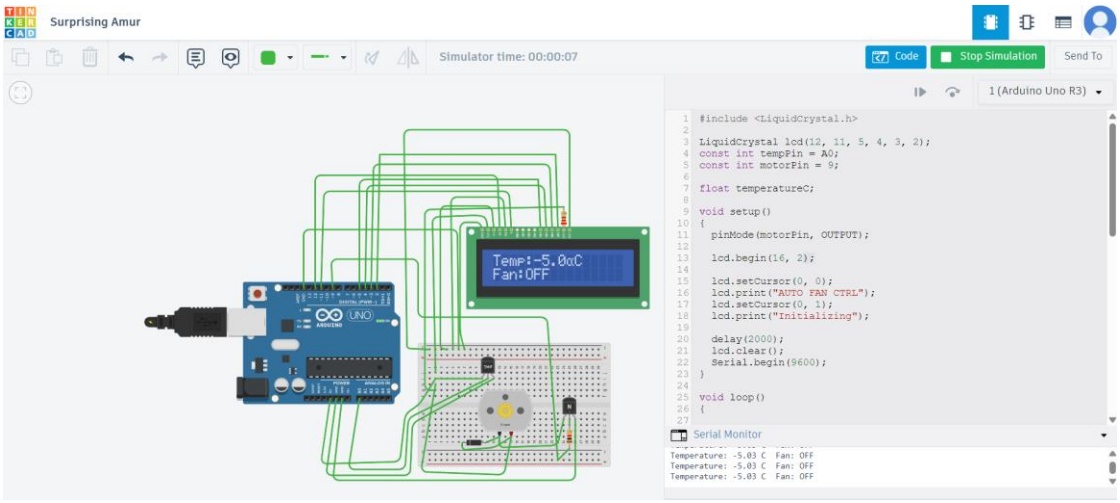


Figure 3. Full source code and Serial Monitor output at -5.0°C — fan correctly held OFF below the first threshold.

Simulation Results

I ran the simulation across a spread of temperatures to confirm the thresholds actually triggered correctly:

- −5.0°C → Fan OFF (cold extreme, well under the first threshold)
- 2.8°C → Fan OFF
- 27.2°C → Fan MEDIUM, PWM 150
- 82.0°C → Fan HIGH, PWM 255 (pushed well past the threshold to confirm it saturates rather than overflowing)

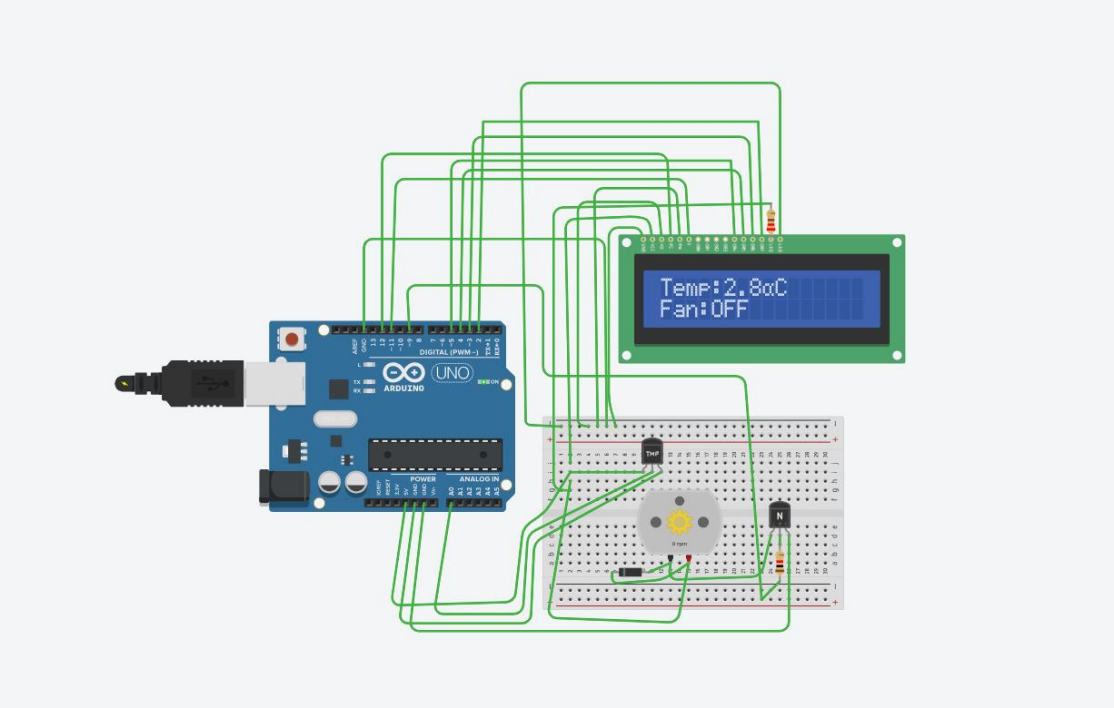


Figure 4. Low temperature test — 2.8°C, Fan: OFF.

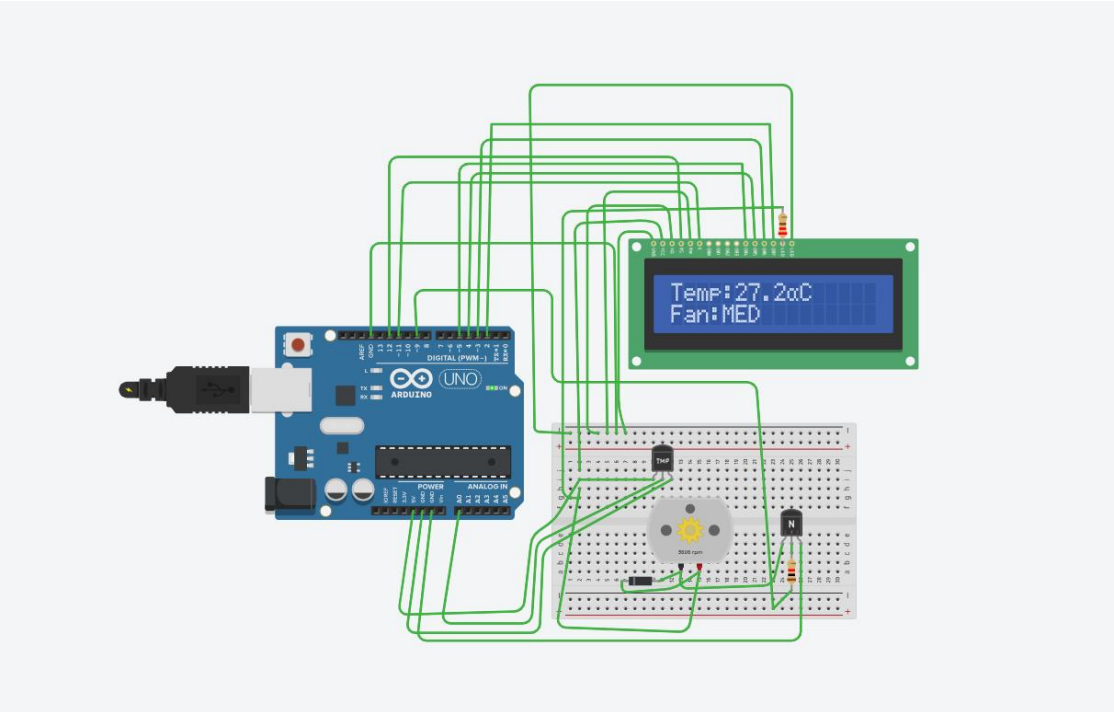


Figure 5. Medium temperature test — 27.2°C, Fan: MEDIUM.

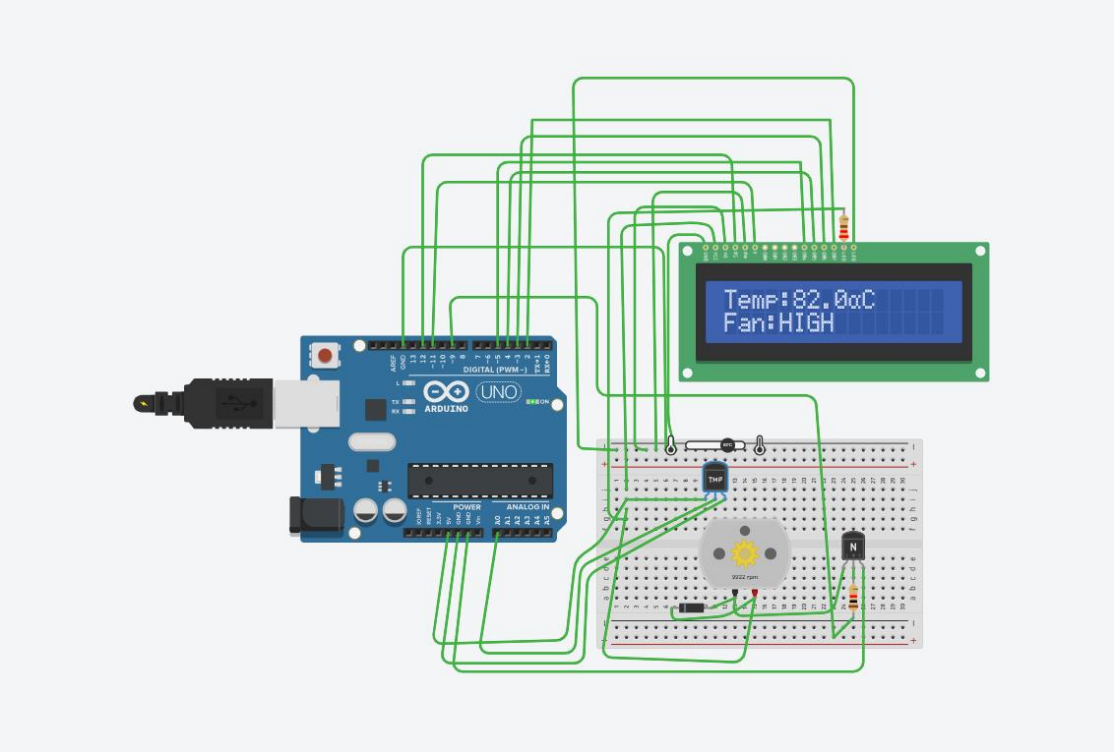


Figure 6. High temperature test — 82.0°C, Fan: HIGH.

All four readings matched the expected PWM output, and the Serial Monitor values stayed consistent with what the LCD displayed, so the analog-to-digital conversion and the threshold logic were both behaving correctly.

One thing worth flagging: the LCD shows “α” instead of “°” in the readout (e.g. “82.0αC” instead of “82.0°C”). That's not a code bug — it's a character-set quirk in how Tinkercad's simulated LiquidCrystal driver maps the degree symbol, since byte 0xDF isn't handled the same way the simulator's font table expects. On real hardware with an actual HD44780 controller, the same code prints the degree symbol correctly, since the physical LCD's character ROM handles that byte differently than the simulator does.

Challenges and What I Learned

The two things that actually needed real thought were the motor driver and the back-EMF protection — both are easy to skip if the goal is just to get something to “work” in a simulator, but they matter once real current is involved. Sizing the base resistor for the transistor and understanding why the flyback diode goes across the motor itself (not in series, not on the supply rail) took more digging than I expected for what looked like a simple add-on component.

The bigger takeaway was noticing how closely this matches the sensor work I've already done with the MPU-6050 and ATmega328P — same idea of reading an analog or digital sensor, running it through some decision logic, and driving an actuator safely. The control loop itself is almost a template; what changes between projects is the sensor, the actuation method, and the thresholds.

Conclusion

The system worked as intended: temperature in, fan response out, with the LCD and Serial Monitor both confirming the logic at every step. As a simulation it's about as basic as a control loop gets, but it covers the same fundamentals — analog sensing, threshold-based decision making, PWM actuation, and protected motor driving — that scale up into more complex embedded and IoT systems. The most natural next step, and one the task description hints at directly, would be swapping the Arduino Uno for an ESP8266/ESP32 and pushing the same temperature data to a service like ThingSpeak — turning this from a closed local loop into something remotely monitorable.

References

1. Arduino Uno Official Documentation.
2. TMP36 Temperature Sensor Datasheet, Analog Devices.
3. Tinkercad Circuits Simulation Platform.
4. Arduino PWM / analogWrite() Reference Documentation.
5. CodeAlpha Internship Task Guidelines.